



សាកលវិទ្យាល័យអង្គរ

ANGKOR UNIVERSITY

បរិញ្ញាបត្រ វិទ្យាសាស្ត្រកុំព្យូទ័រ

ឯកសារ (HANDOUT) - វគ្គទី ១

សេចក្តីផ្តើម .NET Core, VS Code និងការគ្រប់គ្រងកូដតាម GitHub

បច្ចេកវិទ្យា ASP.NET និងប្រព័ន្ធគ្រប់គ្រងមូលដ្ឋានទិន្នន័យ SQL Server



រៀបចំដោយសាស្ត្រាចារ្យ៖

សាស្ត្រាចារ្យ សៀង លីម (SLim)



👤 ព័ត៌មានសាស្ត្រាចារ្យ និងមុខវិជ្ជា



សាស្ត្រាចារ្យបង្រៀន សៀង លីម (Soeurng LIM)

🎓 កម្រិតគុណវុឌ្ឍិ: បរិញ្ញាបត្រជាន់ខ្ពស់ - ព័ត៌មានវិទ្យា

✉ អ៊ីម៉ែល: soeurnglim@gmail.com

☎ ទូរស័ព្ទ: ០៦១ ២២៧ ៩៣៣

★ ក្រេឌីតមុខវិជ្ជា: ៣ ក្រេឌីត (IT401-WebAPI)

📖 មាតិកាជំពូកទី ១ (Chapter Contents)

ចំណុច	ខ្លឹមសារមេរៀន (Topics)
១.១	ពាក្យពេញ (Abbreviations) និងទិដ្ឋភាពទូទៅនៃបច្ចេកវិទ្យា .NET
១.២	ការស្វែងយល់ពី Application Programming Interface (API) ស្តីពីស្រាវជ្រាវ
១.៣	ស្ថាបត្យកម្ម Model-View-Controller (MVC) ពិស្តារ និងលំហូរការងារ
១.៤	ការស្វែងយល់ពីប្រព័ន្ធគ្រប់គ្រងមូលដ្ឋានទិន្នន័យ SQL Server
១.៥	របៀបចូលទៅប្រើប្រាស់ និងរៀបចំបរិស្ថាន (Environment Setup)
១.៦	របៀបចូលទៅប្រើប្រាស់ .NET CLI និងការបង្កើត Project ថ្មី
១.៧	របៀបចូលទៅប្រើប្រាស់កម្មវិធី GitHub Desktop (GUI)

🗨 មុខវិជ្ជានេះត្រូវបានរចនាឡើងយ៉ាងពិសេសសម្រាប់និស្សិតដែលទើបឈានជើងចូលក្នុងពិភពនៃការអភិវឌ្ឍប្រព័ន្ធ **Back-End** និង **Application Programming Interface (API)**។ នៅក្នុងយុគសម័យឌីជីថល កម្មវិធី Web និង Mobile សុទ្ធតែពឹងផ្អែកលើ API ជាស្ថានចម្លងទិន្នន័យយ៉ាងសំខាន់បំផុត។



🎯 គោលបំណងនៃជំពូកទី១ (Chapter Objectives)

បន្ទាប់ពីបញ្ចប់ការសិក្សាជំពូកទី១ នេះ និស្សិតនឹងមានសមត្ថភាព៖

- **យល់ដឹងពីមូលដ្ឋានគ្រឹះ៖** អាចពន្យល់ពីទិដ្ឋភាពទូទៅនៃ .NET Core, ASP.NET Core និងអត្ថប្រយោជន៍របស់វា។
- **ស្គាល់ពី API៖** យល់ច្បាស់ពីតួនាទីរបស់ API ជាស្ថានភាពទំនាក់ទំនងរវាងកម្មវិធី និងទម្រង់ទិន្នន័យ JSON។
- **យល់ពី MVC៖** អាចពន្យល់ពីស្ថាបត្យកម្ម Model-View-Controller (MVC) និងលំហូរការងាររបស់វាក្នុង ASP.NET។
- **រៀបចំឧបករណ៍៖** ចេះរៀបចំ Environment សម្រាប់ការអភិវឌ្ឍន៍ ដោយដំឡើង និងប្រើប្រាស់ .NET SDK, VS Code, C# Dev Kit, SQL Server និង Postman។
- **ប្រើប្រាស់ .NET CLI៖** អាចប្រើប្រាស់ Command Line ដើម្បីបង្កើត ចងក្រង (build) និងដំណើរការ (run) គម្រោង ASP.NET Core Web API ដំបូងរបស់ខ្លួន។
- **គ្រប់គ្រងកូដ៖** យល់ដឹងពីមូលដ្ឋានគ្រឹះនៃ Git និងអាចប្រើប្រាស់ GitHub Desktop ដើម្បីគ្រប់គ្រងកំណែកូដ និងរក្សាទុកគម្រោងលើ Cloud។



Objectives Image



១.១ ទិដ្ឋភាពទូទៅនៃបច្ចេកវិទ្យា .NET Core

ពាក្យពេញ និងអក្សរកាត់សំខាន់ៗ (Abbreviations)

 Abbreviations Image

អក្សរកាត់	ពាក្យពេញ (Full Form)	អត្ថន័យសង្ខេប
API	Application Programming Interface	ស្ថានភាពទំនាក់ទំនងរវាងកម្មវិធីពីរផ្សេងគ្នា
ASP	Active Server Pages	បច្ចេកវិទ្យាសម្រាប់បង្កើត Web/API របស់ Microsoft
MVC	Model-View-Controller	ស្ថាបត្យកម្មបែងចែករចនាសម្ព័ន្ធក្នុង ៣ ផ្នែក
SDK	Software Development Kit	កញ្ចប់ឧបករណ៍ចាំបាច់សម្រាប់សរសេរកូដ
JSON	JavaScript Object Notation	ទម្រង់ទិន្នន័យស្រាល សម្រាប់ការបញ្ជូន Data
CLI	Command Line Interface	ការបញ្ជាប្រាណតាមរយៈការវាយអក្សរ



តើអ្វីទៅជា .NET Core?



What is .NET Core

.NET Core (បច្ចុប្បន្នហៅត្រឹមតែ .NET) គឺជា Platform កូដបើកចំហ (Open-source) ដ៏មានអានុភាពដែលបង្កើតឡើងដោយក្រុមហ៊ុន Microsoft សម្រាប់ប្រើប្រាស់ក្នុងការអភិវឌ្ឍន៍កម្មវិធីគ្រប់ប្រភេទ។ លក្ខណៈពិសេសដែលធ្វើឱ្យវាក្លាយជាជម្រើសចម្បង៖

Cross-Platform: អាចដំណើរការបាននៅលើប្រព័ន្ធប្រតិបត្តិការ Windows, macOS, និង Linux ដោយរលូន។

High Performance: មានល្បឿនលឿនខ្លាំង និងប្រើប្រាស់ Memory កុំព្យូទ័រតិចជាងជំនាន់មុនៗ។

Security: ផ្តល់នូវប្រព័ន្ធសុវត្ថិភាពខ្ពស់បំផុត ដែលស័ក្តិសមសម្រាប់ការអភិវឌ្ឍប្រព័ន្ធធនាគារ។

ប្រវត្តិ និងការវិវត្តន៍ (.NET Framework vs .NET 8)



History Table



អ្វីទៅជា ASP.NET Core?

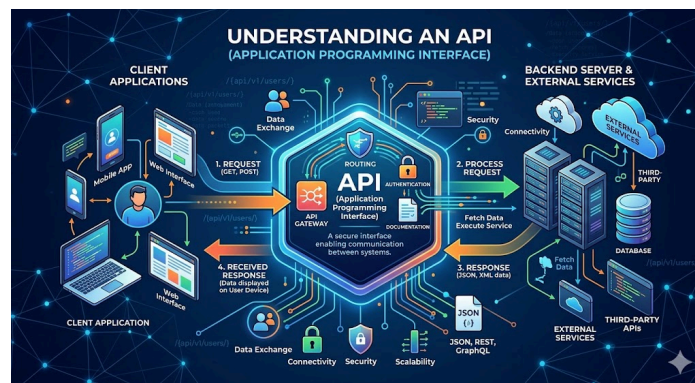


What is ASP.NET Core

ASP.NET Core គឺជា Framework របស់ .NET ដែលត្រូវបានរចនាឡើងយ៉ាងពិសេសសម្រាប់តែការបង្កើត **Web Applications** និង **Application Programming Interface (API)** ប៉ុណ្ណោះ។ វាមានបញ្ចូលនូវឧបករណ៍ស្រាប់ៗ (Built-in Tools) ដូចជា ការគ្រប់គ្រងសុវត្ថិភាព (Authentication), ការកំណត់ផ្លូវ (Routing), និងការភ្ជាប់ជាមួយប្រព័ន្ធមូលដ្ឋានទិន្នន័យ SQL Server បានយ៉ាងងាយស្រួល។

១.២ ការស្វែងយល់ពី Application Programming Interface (API)

API ប្រៀបបានទៅនឹងស្ថានភាពមធ្យមទិន្នន័យ ដែលអនុញ្ញាតឱ្យកម្មវិធី (Software) ពីរផ្សេងគ្នាអាចធ្វើការសន្ទនា និងផ្លាស់ប្តូរទិន្នន័យគ្នាបានយ៉ាងងាយស្រួល។ ក្នុងមេរៀននេះ យើងផ្តោតលើ **RESTful API** ដែលបញ្ជូនទិន្នន័យជាទម្រង់ **JSON** ល្បឿនលឿន។





💡 ករណីសិក្សា៖ អ្នករត់ក្នុងកោដិយដ្ឋាន

ប្រៀបធៀប API ទៅនឹងអ្នករត់ក្នុងកោដិយដ្ឋាន៖

១. ភ្ញៀវ (Client App): អ្នកប្រើប្រាស់ Mobile/Web ដែលត្រូវការទិន្នន័យ។
២. អ្នករត់ក្នុងកោដិយដ្ឋាន (API): ទទួលសំណើ (Request) ពីភ្ញៀវ រួចយកទៅប្រាប់បងកៅនៅខាងក្រោយ។
៣. បងកៅ (Database/Server): រៀបចំទិន្នន័យដែលស្នើសុំ រួចផ្ញើត្រឡប់មកវិញតាមរយៈអ្នករត់ក្នុងកោដិយដ្ឋាន (ជាទម្រង់ Data/JSON Response)។



ប្រភេទនៃ HTTP Methods (RESTful API)



ឯកសារ (Handout) | ជំពូកទី ១

ASP.NET CORE API & SQL SERVER



Method	តួនាទី (Action)	ការពន្យល់សង្ខេប
GET	អាន (Read)	ប្រើសម្រាប់ស្នើសុំទាញយកទិន្នន័យពី Server មកបង្ហាញ។
POST	បញ្ចូល (Create)	បញ្ជូនទិន្នន័យថ្មីទៅរក្សាទុកក្នុង Database (ឧ. បង្កើតគណនី)។
PUT	កែប្រែ (Update)	ធ្វើបច្ចុប្បន្នភាពព័ត៌មានដែលមានស្រាប់ទាំងស្រុង។
DELETE	លុប (Delete)	លុបទិន្នន័យចេញពីប្រព័ន្ធ។

លេខកូដឆ្លើយតបរបស់ API (HTTP Status Codes)

Status Code	អត្ថន័យនៃលេខកូដឆ្លើយតបនីមួយៗ
200 OK	សំណើទទួលបានជោគជ័យ។ ទិន្នន័យត្រូវបានបញ្ជូនត្រឡប់ទៅវិញដោយរលូន។
201 Created	ទិន្នន័យថ្មី (POST) ត្រូវបានបង្កើត និងរក្សាទុកក្នុង Database ជោគជ័យ។
400 Bad Request	សំណើមានកំហុស។ អ្នកប្រើប្រាស់អាចបញ្ជូនទិន្នន័យខុសទម្រង់ ឬខ្វះខាតខ្លះៗ។
404 Not Found	រកមិនឃើញទិន្នន័យ ឬអាសយដ្ឋាន API (URL) ដែលបានស្នើសុំនោះទេក្នុងប្រព័ន្ធ។
500 Internal Error	មានកំហុសកើតឡើងនៅលើ Server (កូដផ្ទៃក្នុង Error) ដែលអ្នកអភិវឌ្ឍន៍ត្រូវជួសជុល។





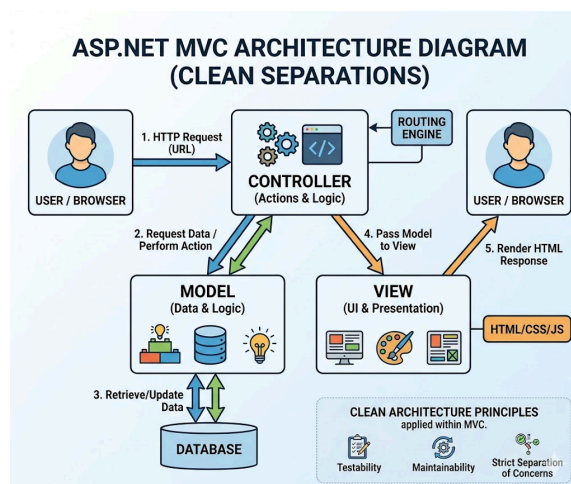
១.៣ ស្ថាបត្យកម្ម Model-View-Controller (MVC)

MVC គឺជាស្ថាបត្យកម្មរចនាសម្ព័ន្ធកូដដ៏ពេញនិយមដែលបែងចែកការងាររបស់កម្មវិធីជា ៣ ផ្នែកធំៗដាច់ពីគ្នា ដើម្បីងាយស្រួលអភិវឌ្ឍន៍ និងថែទាំ៖

Model (M): ជាផ្នែកគ្រប់គ្រងទិន្នន័យ (Data Structure)។ វាទំនាក់ទំនងផ្ទាល់ជាមួយ Database (ឧ. SQL Server) ដើម្បីរក្សាទុក ឬទាញយកទិន្នន័យ។

View (V): ជាផ្នែកបង្ហាញ (User Interface)។ នៅក្នុងការបង្កើត API, View ត្រូវបានជំនួសដោយទម្រង់ទិន្នន័យ **JSON Response** ដែលបញ្ជូនទៅឱ្យ App ផ្សេងទៀតយកទៅបង្ហាញ។

Controller (C): ជាខួរក្បាលអ្នកចាត់ចែង។ ទទួលសំណើ (Request) ពីអ្នកប្រើប្រាស់ -> ពិនិត្យមើល -> បញ្ជាឱ្យ Model ទាញទិន្នន័យ -> យកទិន្នន័យនោះប្រគល់ឱ្យ View។





តើ MVC ធ្វើការជាមួយគ្នាយ៉ាងដូចម្តេច?

1. អ្នកប្រើប្រាស់ធ្វើសកម្មភាព (ឧ: ចុចប៊ូតុងស្វែងរកសិស្ស)។
2. សំណើ (HTTP Request) ត្រូវបានបញ្ជូនទៅកាន់ **Controller**។
3. Controller ពិនិត្យសំណើ រួចបញ្ជូនទៅកាន់ **Model** ឱ្យទៅទាញយកទិន្នន័យក្នុង SQL Database។
4. Model បញ្ជូនទិន្នន័យត្រឡប់មក Controller វិញ។
5. Controller យកទិន្នន័យនោះបំប្លែងជា JSON រួចឆ្លើយតប (Response) ទៅកាន់ **View** (App/Browser)។

🍕 ឧទាហរណ៍: ការកម្ទីង Pizza (MVC Flow)

View: ជាកាតាឡុក ឬអេក្រង់ទូរស័ព្ទដែលអ្នកចុចរើស Pizza។

Controller: ជាអ្នកគិតលុយ ដែលទទួលការកម្ទីងពីអ្នក ហើយស្រែកប្រាប់ទៅចុងភៅ។

Model: ជាចុងភៅ និងទូទឹកកក ដែលមានតួនាទីលាយម្សៅ ដុតនំឱ្យឆ្អិន រួចហុចប្រអប់ Pizza មកឱ្យអ្នកគិតលុយវិញ។

📄 ១.៤ ប្រព័ន្ធគ្រប់គ្រងមូលដ្ឋានទិន្នន័យ SQL Server

SQL Server គឺជាប្រព័ន្ធគ្រប់គ្រងមូលដ្ឋានទិន្នន័យទំនាក់ទំនង (RDBMS) ដ៏រឹងមាំសម្រាប់ក្រុមហ៊ុនធំៗ (Enterprise) ដែលបង្កើតដោយ Microsoft។ វាស្ថិតនៅក្នុងខ្សែអាជីវកម្ម .NET តាមរយៈបច្ចេកវិទ្យា **Entity Framework Core**។





> ១.៥ និង ១.៦ ឧបករណ៍សរសេរកូដ និងការប្រើប្រាស់ .NET CLI

ឧបករណ៍ចាំបាច់សម្រាប់ការអភិវឌ្ឍន៍មាន៖

១. .NET 8.0 SDK	ទាញយកពីវេបសាយ dot.net/download ។ វាមានផ្ទុក Compiler សម្រាប់ Run កូដ C#។
២. Visual Studio Code	ទាញយកពី code.visualstudio.com ។ វាជា IDE ដ៏ស្រាល និងលឿន។ យើងមិនប្រើ VS ធំទេ ដើម្បីសន្សំសំចៃ Memory កុំព្យូទ័រ។
៣. C# Dev Kit	Extension ក្នុង VS Code សម្រាប់ជំនួយការសរសេរកូដ និង Auto-complete។
៤. SQL Server Express	សម្រាប់ផ្ទុកទិន្នន័យ និងទាញយក SSMS សម្រាប់មើលតារាងទិន្នន័យដោយផ្ទាល់។

ឧបករណ៍ជំនួយ៖ Postman (API Testing Tool)

Postman គឺជាកម្មវិធីដ៏ពេញនិយមបំផុតសម្រាប់អ្នកអភិវឌ្ឍន៍ Backend ក្នុងការធ្វើតេស្តសាកល្បង API មុននឹងបញ្ជូនទៅឱ្យអ្នកអភិវឌ្ឍន៍ Frontend ប្រើប្រាស់។



Postman Tool





🔧 លំហាត់អនុវត្ត (Lab 1.1): ការរៀបចំ Environment Setup

ដំឡើង .NET 8.0 SDK, Visual Studio Code (រួមជាមួយ C# Dev Kit Extension) និង GitHub Desktop លើកុំព្យូទ័រដែលបានរៀបចំរួចរាល់ រួចធ្វើការត្រួតពិនិត្យ (Verify) តាមរយៈការវាយបញ្ជា `dotnet --version` ក្នុង Terminal។

ការប្រើប្រាស់ .NET CLI (Command Line Interface)

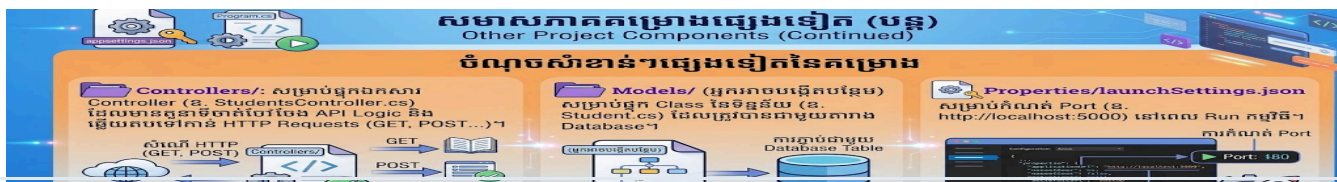
ជាការបញ្ជាក់កុំឱ្យភ្លេចតាមរយៈការវាយអក្សរក្នុង Terminal របស់ VS Code:

Command	មុខងារ
<code>dotnet --version</code>	ពិនិត្យមើលជំនាន់របស់ .NET SDK ដែលបានដំឡើង។
<code>dotnet new webapi -n</code>	បង្កើតគម្រោង ASP.NET Core API ថ្មីដោយស្វ័យប្រវត្តិ។
<code>dotnet build</code>	ចងក្រងកូដ (Compile) ដើម្បីពិនិត្យរកមើលកំហុស (Errors)។
<code>dotnet run</code>	ដំណើរការកម្មវិធីឱ្យចេញជា Web Server ជាក់ស្តែងនៅលើ Localhost។

</> លំហាត់អនុវត្ត (Lab 1.2 & 1.3): ការបង្កើត API & C# Controller

- ប្រើបញ្ជា `dotnet new webapi -n SchoolSystem` ដើម្បីបង្កើតគម្រោងថ្មី។
- បង្កើត `TestController.cs` រួចសរសេរកូដ C# ឱ្យ Return ឈ្មោះ និងលេខសម្គាល់និស្សិតរបស់អ្នកជាទម្រង់ JSON នៅពេល `dotnet run`។

រចនាសម្ព័ន្ធ Project Directory សំខាន់ៗ



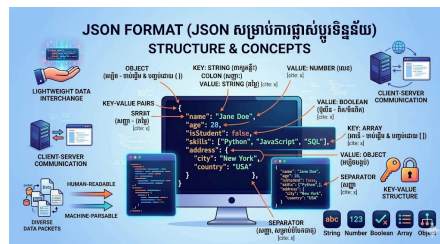


ឧទាហរណ៍៖ ការសរសេរ Controller ដំបូង (C#)

ខាងក្រោមនេះគឺជាកូដ C# សម្រាប់បង្កើត Controller សាមញ្ញមួយដែលអាចបញ្ចេញទិន្នន័យ JSON បាន៖

```
using Microsoft.AspNetCore.Mvc;  
  
namespace SchoolApi.Controllers {  
    [ApiController]  
    [Route("api/[controller]")]  
    public class StudentsController : ControllerBase {  
  
        // ទទួល Request ជាប្រភេទ GET ពី Client  
        [HttpGet]  
        public IActionResult GetStudent() {  
            // Return ទិន្នន័យជាទម្រង់ JSON ជាមួយ Status 200 OK  
            return Ok(new { Id = 1, Name = "Soeurng LIM", Major = "CS" });  
        }  
    }  
}
```

</> អ្វីទៅជា JSON Format និងការប្រើប្រាស់ API?



អត្ថប្រយោជន៍ **JSON**: ជាទម្រង់ទិន្នន័យស្តង់ដារ ស្រាល និងងាយស្រួលអានដោយប្រើរចនាសម្ព័ន្ធ (Key-Value)។



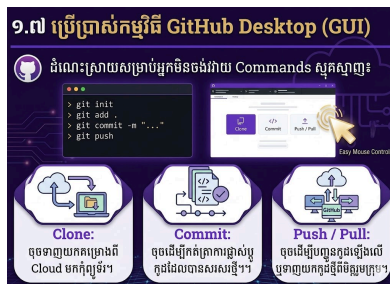
API តែមួយប្រើបានគ្រប់ឧបករណ៍: Web, iOS, Android ជួយសន្សំសំចៃពេលវេលាអភិវឌ្ឍន៍។





១.៧ ការគ្រប់គ្រងកូដជាមួយ Git និង GitHub

ការសរសេរកូដប្រកបដោយស្តង់ដារវិជ្ជាជីវៈ ទាមទារឱ្យមានការគ្រប់គ្រងប្រវត្តិឯកសារកូដឱ្យបានច្បាស់លាស់ដើម្បីងាយស្រួលធ្វើការងារជាក្រុម។



Git: ជាឧបករណ៍ក្នុងម៉ាស៊ីនកុំព្យូទ័រសម្រាប់កត់ត្រាលំដាប់ការផ្លាស់ប្តូរកូដ (Version Control System) ការពារការបាត់បង់កូដ។

GitHub: ជាប្លង់ផ្ទុកកូដនៅលើ Internet (Cloud Platform) ដែលជួយសម្រួលដល់ការធ្វើការងារជាក្រុម (Collaboration)។

GitHub Desktop: ជាកម្មវិធី (GUI) ដែលជួយឱ្យការប្រើប្រាស់ Git កាន់តែងាយស្រួលដោយគ្រាន់តែប្រើ Mouse ចុច។ មុខងារសំខាន់ៗ៖

Clone: ទាញយកគម្រោងពី Cloud មកម៉ាស៊ីនកុំព្យូទ័រ។

Commit: កត់ត្រាការផ្លាស់ប្តូរកូដដែលបានសរសេរថ្មីៗ។

Push / Pull: បញ្ជូនកូដឡើងទៅកាន់ Cloud ដើម្បីរក្សាទុក ឬចែករំលែកជាមួយមិត្តរួមក្រុម។

📁 លំហាត់អនុវត្ត (Lab 1.4)៖ ការប្រើប្រាស់ GitHub Cloud Sync

ភ្ជាប់គម្រោងរបស់អ្នកទៅកាន់ GitHub ដោយការបង្កើត Repository ថ្មីឈ្មោះ 'ASPNet-Lab1' រួចប្រើប្រាស់កម្មវិធី **GitHub Desktop** ដើម្បី Commit និង Push កូដនោះទៅកាន់ Cloud។





📖 សទ្ទានុក្រម (Glossary)

ពាក្យបច្ចេកទេសសំខាន់ៗដែលនិស្សិតត្រូវស្វែងយល់ និងចងចាំនៅក្នុងជំពូកទី ១ នេះ៖

ពាក្យបច្ចេកទេស	ការពន្យល់ជាភាសាខ្មែរ
SDK	កញ្ចប់ឧបករណ៍ចាំបាច់សម្រាប់សរសេរ ចងក្រង និងដំណើរការកូដកម្មវិធី។
API	ស្ថានភាពទំនាក់ទំនងរវាងកម្មវិធីពីរផ្សេងគ្នា អនុញ្ញាតឱ្យពួកវាសន្ទនាគ្នាបាន។
JSON	ទម្រង់ទិន្នន័យអត្ថបទស្រាល និងពេញនិយមបំផុតសម្រាប់ការបញ្ជូន Data រវាង Client និង Server។
RESTful	ស្តង់ដាររចនា API ដែលប្រើប្រាស់ HTTP Methods ដូចជា (GET, POST, PUT, DELETE) ជាដើម។
MVC	ស្ថាបត្យកម្មបែងចែករចនាសម្ព័ន្ធកូដជា ៣ ផ្នែក (ទិន្នន័យ, ផ្ទាំងបង្ហាញ, អ្នកបញ្ជា)។
CLI	ការបញ្ជាប្រាណតាមរយៈការវាយអក្សរជាបញ្ជា (Commands) ជំនួសការប្រើ Mouse។
GUI	ការបញ្ជាប្រាណតាមរយៈការចុច Mouse លើរូបតំណាង (Icons) ឬប៊ូតុង។
RDBMS	ប្រព័ន្ធគ្រប់គ្រងមូលដ្ឋានទិន្នន័យទំនាក់ទំនង (ឧទាហរណ៍ដូចជា SQL Server ជាដើម)។
Repository (Repo)	ឃ្លាំងសម្រាប់ផ្ទុកកូដ និងប្រវត្តិការផ្លាស់ប្តូរកូដនៅលើប្រព័ន្ធ GitHub។
Commit	ការកត់ត្រាទុកនូវរាល់ការផ្លាស់ប្តូរកូដនីមួយៗទៅក្នុង Local History នៃ Git។





ឯកសារយោង និងគន្ថនិទ្ទេស (References)

Microsoft Learn (ASP.NET Core Docs): learn.microsoft.com/en-us/aspnet/core/

Microsoft Learn (.NET CLI): learn.microsoft.com/en-us/dotnet/core/tools/

Mozilla Developer Network (JSON): developer.mozilla.org/en-US/docs/Learn/JavaScript/Objects/JSON

GitHub Desktop Documentation: docs.github.com/en/desktop

Freeman, A. (2024). *Pro ASP.NET Core 8*. Apress. (10th Edition)

~ បញ្ចប់ឯកសារឯកសារជំពូកទី ១ ~





១. សំណួរ ខុស/ត្រូវ (១០ សំណួរ)

សេចក្តីណែនាំ៖ សូមគូសសញ្ញា (✓) ប្រសិនបើប្រយោគខាងក្រោមត្រឹមត្រូវ ឬសញ្ញា (✗) ប្រសិនបើខុស។

1. .NET Core អាចដំឡើងបាននៅលើប្រព័ន្ធប្រតិបត្តិការ Windows ប៉ុណ្ណោះ។
2. API ដើរតួជាស្ថានភាពទំនាក់ទំនងរវាងកម្មវិធីពីរផ្សេងគ្នា អនុញ្ញាតឱ្យពួកវាសន្ទនាគ្នាបាន។
3. នៅក្នុងទម្រង់ស្ថាបត្យកម្ម MVC, View ទទួលបន្ទុកក្នុងការគ្រប់គ្រងទិន្នន័យ និងភ្ជាប់ទៅ Database។
4. JSON គឺជាទម្រង់ទិន្នន័យដែលមានទម្ងន់ស្រាល និងពេញនិយមបំផុតសម្រាប់ការបញ្ជូន Data ក្នុង RESTful API។
5. HTTP Method **GET** ត្រូវបានប្រើប្រាស់សម្រាប់បញ្ជាឱ្យទទួលបានទិន្នន័យចេញពី Server។
6. **dotnet run** គឺជាបញ្ជាសម្រាប់ដំឡើងកម្មវិធី ASP.NET Core ឱ្យចេញជា Web Server។
7. SQL Server គឺជាប្រព័ន្ធគ្រប់គ្រងមូលដ្ឋានទិន្នន័យប្រភេទ RDBMS។
8. GitHub Desktop គឺជាកម្មវិធី (GUI) ដែលជួយសម្រួលដល់ការប្រើប្រាស់ Git ដោយមិនចាំបាច់វាយ Commands ស្មុគស្មាញ។
9. ឯកសារ **appsettings.json** ត្រូវបានប្រើប្រាស់សម្រាប់ផ្ទុកកូដ C# នៃ Controller។
10. នៅក្នុង RESTful API, **POST** method ត្រូវបានប្រើដើម្បីបញ្ជូនបង្កើតទិន្នន័យថ្មី។





២. សំណួរ ពហុជ្រើសរើស (ផ្នែកទី ១)

1. តើអក្សរកាត់ API តំណាងឱ្យអ្វី?

- A. A. Application Programming Interface
- B. B. Active Programming Internet
- C. C. Application Process Integration
- D. D. Automated Program Interface

2. នៅក្នុងស្ថាបត្យកម្ម MVC តើ Controller មានតួនាទីអ្វី?

- A. A. រក្សាទុកទិន្នន័យទៅក្នុង Database
- B. B. ទទួលសំណើ (Request) និងបញ្ជាទៅ Model និង View
- C. C. បង្ហាញផ្ទាំង User Interface ដល់អ្នកប្រើប្រាស់
- D. D. គ្រប់គ្រងសុវត្ថិភាពទិន្នន័យតែមួយមុខ

3. តើ Command មួយណាត្រូវបានប្រើសម្រាប់បង្កើតគម្រោង Web API ថ្មី?

- A. A. dotnet create webapi
- B. B. dotnet init webapi
- C. C. dotnet new webapi
- D. D. dotnet start webapi

4. តើទម្រង់ទិន្នន័យមួយណាដែល RESTful API ភាគច្រើនប្រើប្រាស់ដើម្បីបញ្ជូនទិន្នន័យ?

- A. A. JSON
- B. B. HTML
- C. C. TXT
- D. D. CSS

5. តើ HTTP Method មួយណាត្រូវបានប្រើសម្រាប់ធ្វើបច្ចុប្បន្នភាព (Update) ទិន្នន័យទាំងស្រុង?

- A. A. POST
- B. B. PUT
- C. C. GET
- D. D. UPDATE







៣. សំណួរផ្ទៀងផ្ទាត់ (១០ សំណួរ)

សេចក្តីណែនាំ៖ សូមផ្ដល់ចម្លើយតាមរយៈការបំពេញតារាងខាងក្រោម (១-១០) ទៅនឹងការពន្យល់នៅជួរឈរខាងស្តាំ (A-J) ឱ្យបានត្រឹមត្រូវ។

១. GET	
២. POST	
៣. PUT	
៤. DELETE	
៥. Model	
៦. View	
៧. Controller	
៨. JSON	
៩. git push	
១០. dotnet build	
A. សម្រាប់បញ្ចូលទិន្នន័យថ្មីចូលក្នុងប្រព័ន្ធ	
B. ទម្រង់ទិន្នន័យអត្ថបទស្រាវជ្រាវ	
C. គ្រប់គ្រងទិន្នន័យ និងតភ្ជាប់ជាមួយ Database	
D. សម្រាប់ចងក្រងកូដ និងពិនិត្យរក Error	
E. សម្រាប់លុបទិន្នន័យចោល	
F. សម្រាប់ទាញយកទិន្នន័យមកបង្ហាញ	
G. បញ្ជូនកូដឡើងទៅរក្សាទុកក្នុង Cloud (GitHub)	
H. ខ្នាតកម្រិតអ្នកចាត់ចែងសំណើ និងបញ្ហាការងារ	
I. សម្រាប់ធ្វើបច្ចុប្បន្នភាព (កែប្រែ) ទិន្នន័យទាំងស្រុង	





៤. សំណួរបកស្រាយ (៥ សំណួរ)

1. ចូរពន្យល់ពីភាពខុសគ្នារវាង .NET Framework (ជំនាន់ចាស់) និង .NET Core / .NET 8 (ជំនាន់ថ្មី) ឱ្យបានយ៉ាងតិច ៣ ចំណុច។
2. ចូរពន្យល់ពីតួនាទីរបស់ API នៅក្នុងការអភិវឌ្ឍន៍កម្មវិធី ដោយលើកឧទាហរណ៍ជាក់ស្តែងណាមួយ (ឧទាហរណ៍៖ អ្នករត់តុក្កង ភោជនីយដ្ឋាន) មកបញ្ជាក់។
3. តើស្ថាបត្យកម្ម MVC គឺជាអ្វី? ចូរពន្យល់ពីតួនាទី និងមុខងាររបស់ Model, View និង Controller នីមួយៗ។
4. ហេតុអ្វីបានជានៅក្នុងការអភិវឌ្ឍកម្មវិធីជាក្រុម (Team Collaboration) យើងចាំបាច់ត្រូវប្រើប្រាស់ Git និង GitHub?
5. ចូរប្រៀបធៀបបញ្ជា (Commands) សំខាន់ៗទាំង ៣ ដើម្បីបង្កើតគម្រោង ASP.NET Core Web API ថ្មីមួយ ចងក្រងកូដ និងដំណើរការវាជា Local Server។





🔑 អត្រាកំណែ (Answer Keys)

១. ចម្លើយ ខុស/ត្រូវ

១	២	៣	៤	៥
ខុស	ត្រូវ	ខុស	ត្រូវ	ខុស
៦	៧	៨	៩	១០
ត្រូវ	ត្រូវ	ត្រូវ	ខុស	ត្រូវ

២. ចម្លើយ ពហុជ្រើសរើស

១	២	៣	៤	៥
A	B	C	A	B
៦	៧	៨	៩	១០
A	D	C	B	C

៣. ចម្លើយ ផ្គុំផ្គង

១	២	៣	៤	៥	៦	៧	៨	៩	១០
F	A	I	E	C	J	H	B	G	D

៤. ចម្លើយ សំណួរបកស្រាយ (សង្ខេប)

- ១. ភាពខុសគ្នា: .NET Core គឺជា Cross-Platform (អាចដំឡើងលើ Windows, Linux, macOS), ជា Open-source, និងមានល្បឿនលឿនជាង ក៏ដូចជាប្រើប្រាស់ធនធានកុំព្យូទ័រ (Memory) តិចជាង .NET Framework ជំនាន់ចាស់។
- ២. តួនាទី API: API គឺជាស្ថានភាពទំនាក់ទំនងកម្មវិធី២។ ប្រៀបដូចជាអ្នករត់តុ ដែលយក Request ពីក្រៅ(App) ទៅប្រាប់ ចុងកៅ(Server/Database) រួចបញ្ជូនទិន្នន័យ Data/JSON (អាហារ) ត្រឡប់មកឱ្យភ្ញៀវវិញ។
- ៣. MVC: MVC ជាស្ថាបត្យកម្មកូដចែកជា៣ផ្នែក។ **Model:** គ្រប់គ្រងទិន្នន័យ។ **View:** ផ្ទាំងបង្ហាញ UI ឬ JSON Output។ **Controller:** អ្នកចាត់ចែងសំណើ ទទួល Request ពី Client បញ្ជា Model រួចបញ្ជូនទៅ View បង្ហាញ។
- ៤. ហេតុអ្វីប្រើ Git/GitHub: Git ជួយកាត់ត្រាប្រវត្តិនៃការផ្លាស់ប្តូរកូដនីមួយៗ ការពារការបាត់បង់កូដ។ GitHub អនុញ្ញាតឱ្យផ្ទុកកូដលើ Cloud ងាយស្រួលដល់ក្រុម Developers អាចចែករំលែកកូដគ្នាដោយសុវត្ថិភាព។
- ៥. ជំហានបង្កើត Project: បង្កើត: `dotnet new webapi -n MyProject` | ចងក្រង: `dotnet build` | ដំឡើង: `dotnet run`



